

08_metrics_dashboard

February 16, 2026

1 08 — Metrics dashboard

Purpose: Visualize performance, predictive multiplicity, and spatial multiplicity metrics for the Rashomon set. **Aggregates across all runs** (seeds) with k=30 for spatial metrics.

1.1 Imports and setup

```
[43]: import sys
      from pathlib import Path

      ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
      if str(ROOT) not in sys.path:
          sys.path.insert(0, str(ROOT))
      if str(ROOT / "src") not in sys.path:
          sys.path.insert(0, str(ROOT / "src"))

      import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
      from scipy import sparse as sp

      from sklearn.metrics import brier_score_loss

      from analysis.experiment_runner import _get_run_dirs
      from analysis.preprocessing import get_transformed_test_features
      from analysis.run_analysis import (
          load_meta,
          load_P_test,
          load_split,
          select_rashomon_global,
          compute_multiplicity_metrics,
          run_spatial,
      )
      from data import load_dataset

      RESULTS_DIR = ROOT / "results"
      K = 25
      K_NN = 30
```

```
EPSILON = 0.05
```

1.2 Load all runs and compute metrics per run

For each run: load meta, P_test, split; get X_test and y_test; select Rashomon (top-K); compute performance, predictive multiplicity, and spatial metrics. Store per-run results for aggregation.

```
[44]: DATASET = "compas"
dataset_dir = RESULTS_DIR / DATASET
run_dirs = _get_run_dirs(dataset_dir)
if not run_dirs:
    raise SystemExit(f"No run dirs in {dataset_dir}")

X, y, _ = load_dataset(DATASET)
records = []
K_values = [5, 10, 15, 20, 25, 30, 40, 50, 60, 70, 80, 90, 100, 250]  #_
    ↪Rashomon set sizes for sensitivity plot
records_k = []

for run_dir in run_dirs:
    meta = load_meta(run_dir)
    P_test_r = load_P_test(run_dir)
    split = load_split(run_dir)
    n_cand = len(meta)
    K_actual = min(K, n_cand)
    idx = select_rashomon_global(run_dir, K=K_actual)
    P_sel = P_test_r[idx]
    y_test = y.iloc[split["test"]].values
    X_test = get_transformed_test_features(run_dir, DATASET)
    if hasattr(X_test, "values"):
        X_test = np.asarray(X_test)
    else:
        X_test = np.asarray(X_test)

    # Performance
    acc_per_model = []
    brier_per_model = []
    for m in range(P_sel.shape[0]):
        p = P_sel[m]
        y_pred = (p >= 0.5).astype(int)
        acc_per_model.append(np.mean(y_pred == y_test))
        brier_per_model.append(brier_score_loss(y_test, p))
    P_ens = P_sel.mean(axis=0)
    acc_ensemble = np.mean((P_ens >= 0.5).astype(int) == y_test)
    brier_ensemble = brier_score_loss(y_test, P_ens)

    # Predictive multiplicity
```

```

mult = compute_multiplicity_metrics(P_sel, epsilon=EPSILON)
pointwise_var = mult["pointwise_variance"]

# Multiplicity metrics for increasing k (for sensitivity plot)
for k in K_values:
    if k <= n_cand:
        idx_k = select_rashomon_global(run_dir, K=k)
        P_sel_k = P_test_r[idx_k]
        mult_k = compute_multiplicity_metrics(P_sel_k, epsilon=EPSILON)
        records_k.append({
            "run": run_dir.name,
            "k": k,
            "mean_variance": mult_k["mean_variance"],
            "ambiguity": mult_k["ambiguity"],
            "disagreement_rate": mult_k["disagreement_rate"],
            "discrepancy": mult_k["discrepancy"],
        })

# Spatial
spatial = run_spatial(run_dir, X_test, K=K_actual, k=K_NN)
W_pysal = spatial["W"]
W = W_pysal.to_sparse() if hasattr(W_pysal, "to_sparse") else W_pysal.sparse
if not isinstance(W, sp.csr_matrix):
    W = sp.csr_matrix(W)
n_test = P_sel.shape[1]
consensus = (P_sel.mean(axis=0) >= 0.5).astype(int)
agreement_per_point = np.zeros(n_test)
local_avg_var = np.zeros(n_test)
for i in range(n_test):
    ne = W[i].indices
    if len(ne) == 0:
        agreement_per_point[i] = 1.0
        local_avg_var[i] = pointwise_var[i]
    else:
        agreement_per_point[i] = np.mean(consensus[ne] == consensus[i])
        local_avg_var[i] = np.mean(pointwise_var[ne])

records.append({
    "run": run_dir.name,
    "acc_mean": np.mean(acc_per_model),
    "acc_std": np.std(acc_per_model, ddof=1) if len(acc_per_model) > 1 else
↪0.0,
    "brier_mean": np.mean(brier_per_model),
    "brier_std": np.std(brier_per_model, ddof=1) if len(brier_per_model) >
↪1 else 0.0,
    "acc_ensemble": acc_ensemble,
    "brier_ensemble": brier_ensemble,

```

```

        "mean_variance": mult["mean_variance"],
        "ambiguity": mult["ambiguity"],
        "disagreement_rate": mult["disagreement_rate"],
        "discrepancy": mult["discrepancy"],
        "pointwise_var_mean": float(np.mean(pointwise_var)),
        "moran_i": spatial["moran_i"],
        "n_hh": int(np.sum(spatial["HH_mask"])),
        "n_ll": int(np.sum(spatial["LL_mask"])),
        "neighborhood_agreement": float(np.mean(agreement_per_point)),
        "lcae": float(np.mean(local_avg_var)),
    })

df_runs = pd.DataFrame(records)
df_k = pd.DataFrame(records_k) if records_k else None
print(f"Aggregated over {len(df_runs)} runs. Rashomon K={K_actual}, \
      ↪k_nn={K_NN}")
if df_k is not None:
    print(f"Metrics vs k: {sorted(df_k['k'].unique())} (n_runs per k: {df_k.\
      ↪groupby('k').size().to_dict()})")
df_runs

```

Aggregated over 10 runs. Rashomon K=25, k_nn=30

Metrics vs k: [np.int64(5), np.int64(10), np.int64(15), np.int64(20),
np.int64(25), np.int64(30), np.int64(40), np.int64(50), np.int64(60),
np.int64(70), np.int64(80), np.int64(90), np.int64(100), np.int64(250)] (n_runs
per k: {5: 10, 10: 10, 15: 10, 20: 10, 25: 10, 30: 10, 40: 10, 50: 10, 60: 10,
70: 10, 80: 10, 90: 10, 100: 10, 250: 10})

```

[44]:      run  acc_mean  acc_std  brier_mean  brier_std  acc_ensemble  \
0  seed=0  0.676757  0.003743  0.213855  0.001216  0.679834
1  seed=1  0.681580  0.003781  0.210165  0.001116  0.690229
2  seed=2  0.689841  0.003431  0.205342  0.000653  0.692308
3  seed=3  0.676784  0.004717  0.209259  0.001401  0.681913
4  seed=4  0.689286  0.005401  0.202550  0.001276  0.685378
5  seed=5  0.684934  0.005583  0.209682  0.001084  0.689536
6  seed=6  0.679113  0.003804  0.208720  0.001102  0.679834
7  seed=7  0.669605  0.005726  0.214295  0.000934  0.670132
8  seed=8  0.683548  0.004196  0.205801  0.001178  0.685378
9  seed=9  0.664948  0.003454  0.212179  0.001497  0.666667

      brier_ensemble  mean_variance  ambiguity  disagreement_rate  discrepancy  \
0      0.213014      0.000841  0.107190      0.163876  0.054082
1      0.208641      0.001524  0.119765      0.266778  0.070470
2      0.204501      0.000841  0.104533      0.169751  0.058478
3      0.208113      0.001145  0.129062      0.286355  0.063741
4      0.201134      0.001416  0.140793      0.276757  0.072128
5      0.208386      0.001295  0.121314      0.238006  0.059298

```

6	0.207264	0.001455	0.131112	0.280273	0.066058
7	0.212569	0.001725	0.142425	0.336928	0.075583
8	0.204159	0.001642	0.133968	0.327727	0.072598
9	0.210983	0.001196	0.118975	0.205865	0.058600

	pointwise_var_mean	moran_i	n_hh	n_ll	neighborhood_agreement	lcae
0	0.000841	0.140503	48	603	0.848395	0.000671
1	0.001524	0.211425	104	547	0.846339	0.001366
2	0.000841	0.205800	55	576	0.843913	0.000683
3	0.001145	0.360500	160	569	0.860869	0.001046
4	0.001416	0.110097	21	483	0.857427	0.001240
5	0.001295	0.140545	46	497	0.850450	0.001094
6	0.001455	0.175005	102	597	0.844329	0.001258
7	0.001725	0.215078	112	535	0.849711	0.001557
8	0.001642	0.291669	137	543	0.854655	0.001508
9	0.001196	0.142169	22	540	0.836637	0.000963

1.3 1. Performance: Accuracy and Brier

Per Rashomon model: accuracy and Brier on test. Optionally: ensemble (mean prediction) accuracy and Brier.

```
[45]: perf_agg = df_runs.agg({
    "acc_mean": ["mean", "std"],
    "brier_mean": ["mean", "std"],
    "acc_ensemble": ["mean", "std"],
    "brier_ensemble": ["mean", "std"],
}).round(4)
print("Performance aggregated across runs (mean ± std over runs):")
print(perf_agg)
print("\nRashomon (mean over models per run): accuracy and Brier averaged_
    across runs.")
print("Ensemble: mean prediction per run, then averaged across runs.")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3.5))
ax1.boxplot(
    [df_runs["acc_mean"], df_runs["acc_ensemble"]],
    tick_labels=["Rashomon (mean/run)", "Ensemble"],
)
ax1.set_ylabel("Accuracy")
ax1.set_title("Accuracy per run")
ax2.boxplot(
    [df_runs["brier_mean"], df_runs["brier_ensemble"]],
    tick_labels=["Rashomon (mean/run)", "Ensemble"],
)
ax2.set_ylabel("Brier")
ax2.set_title("Brier per run")
```

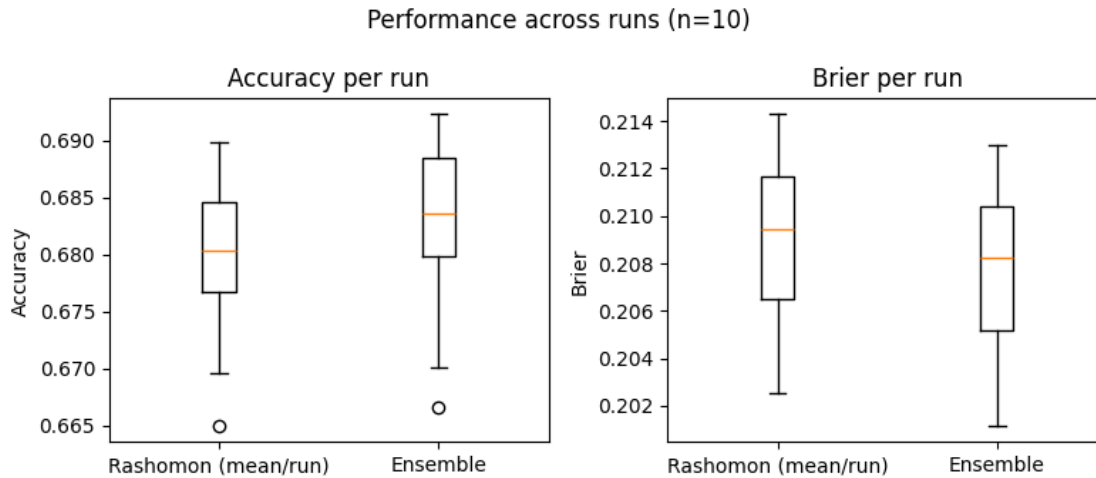
```
plt.suptitle("Performance across runs (n=%d)" % len(df_runs))
plt.tight_layout()
plt.show()
```

Performance aggregated across runs (mean $\pm$ std over runs):

	acc_mean	brier_mean	acc_ensemble	brier_ensemble
mean	0.6796	0.2092	0.6821	0.2079
std	0.0080	0.0038	0.0084	0.0038

Rashomon (mean over models per run): accuracy and Brier averaged across runs.

Ensemble: mean prediction per run, then averaged across runs.



1.4 2. Predictive multiplicity

Mean variance, ambiguity, disagreement rate ($=0.05$), discrepancy. Optionally: distribution of pointwise variance.

```
[46]: mult_cols = ["mean_variance", "ambiguity", "disagreement_rate", "discrepancy"]
mult_agg = df_runs[mult_cols].agg(["mean", "std"]).T.round(4)
mult_agg.columns = ["mean", "std"]
mult_agg["mean  $\pm$  std"] = mult_agg.apply(lambda r: f"{r['mean']:.4f}  $\pm$  {r['std']:.4f}", axis=1)
print("Predictive multiplicity aggregated across runs:")
display(mult_agg[["mean", "std", "mean  $\pm$  std"]])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5))
x = np.arange(len(mult_cols))
means = df_runs[mult_cols].mean().values
stds = df_runs[mult_cols].std().values
ax1.barh(mult_cols, means, xerr=stds, color="steelblue", capsize=3)
ax1.set_xlabel("Value (mean  $\pm$  std over runs)")
```

```

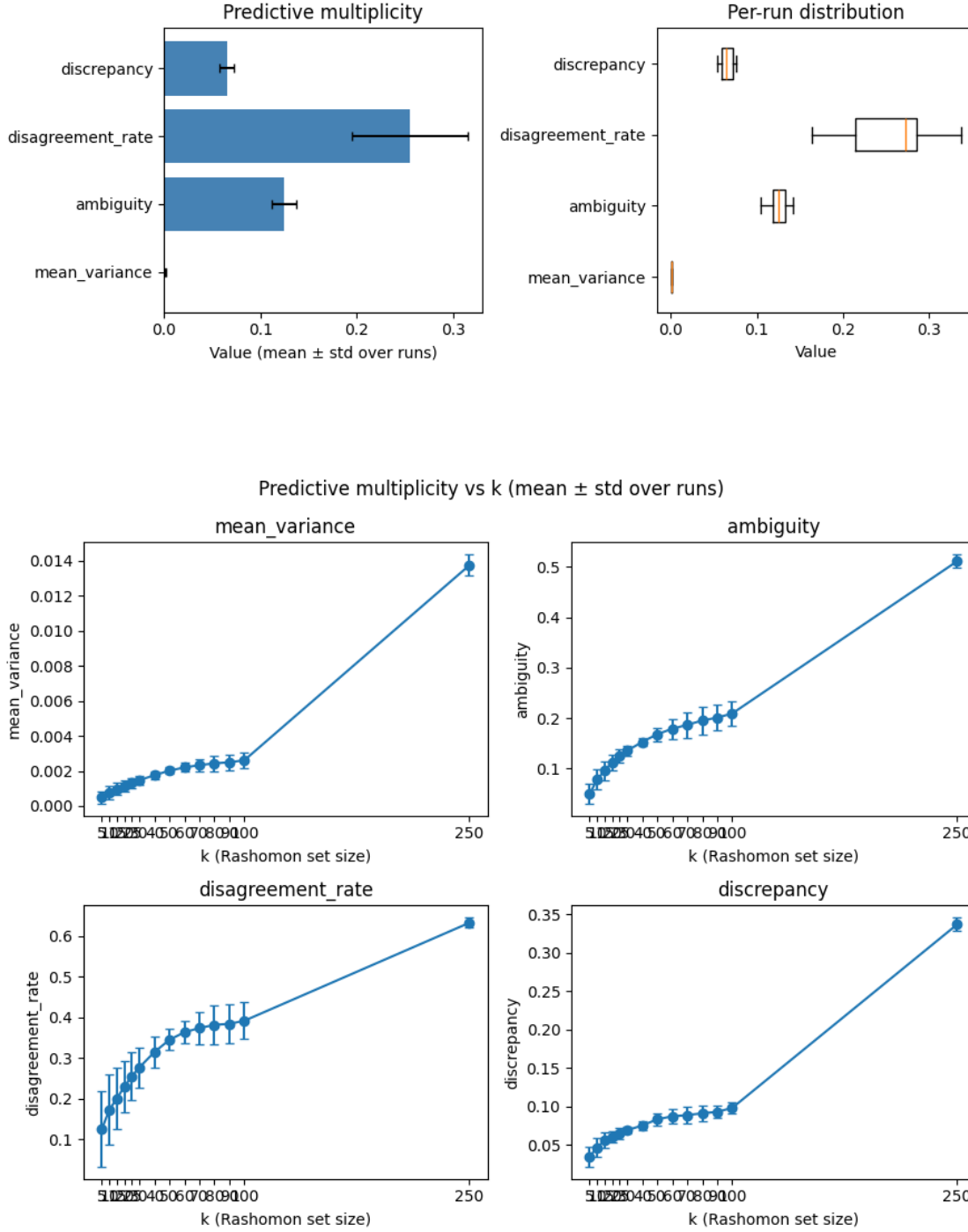
ax1.set_title("Predictive multiplicity")
ax2.boxplot(
    [df_runs["mean_variance"], df_runs["ambiguity"],
    df_runs["disagreement_rate"], df_runs["discrepancy"]],
    tick_labels=mult_cols,
    vert=False,
)
ax2.set_xlabel("Value")
ax2.set_title("Per-run distribution")
plt.tight_layout()
plt.show()

# Metrics vs increasing k (Rashomon set size)
if df_k is not None and len(df_k) > 0:
    k_agg = df_k.groupby("k")[mult_cols].agg(["mean", "std"]).reset_index()
    fig2, axes = plt.subplots(2, 2, figsize=(9, 7))
    axes = axes.flatten()
    for i, col in enumerate(mult_cols):
        ax = axes[i]
        k_vals = k_agg["k"].values
        means = k_agg[(col, "mean")].values
        stds = k_agg[(col, "std")].values
        ax.errorbar(k_vals, means, yerr=stds, marker="o", capsize=3)
        ax.set_xlabel("k (Rashomon set size)")
        ax.set_ylabel(col)
        ax.set_title(col)
        ax.set_xticks(k_vals)
    plt.suptitle("Predictive multiplicity vs k (mean ± std over runs)")
    plt.tight_layout()
    plt.show()

```

Predictive multiplicity aggregated across runs:

	mean	std	mean ± std
mean_variance	0.0013	0.0003	0.0013 ± 0.0003
ambiguity	0.1249	0.0129	0.1249 ± 0.0129
disagreement_rate	0.2552	0.0601	0.2552 ± 0.0601
discrepancy	0.0651	0.0074	0.0651 ± 0.0074



1.5 3. Spatial multiplicity

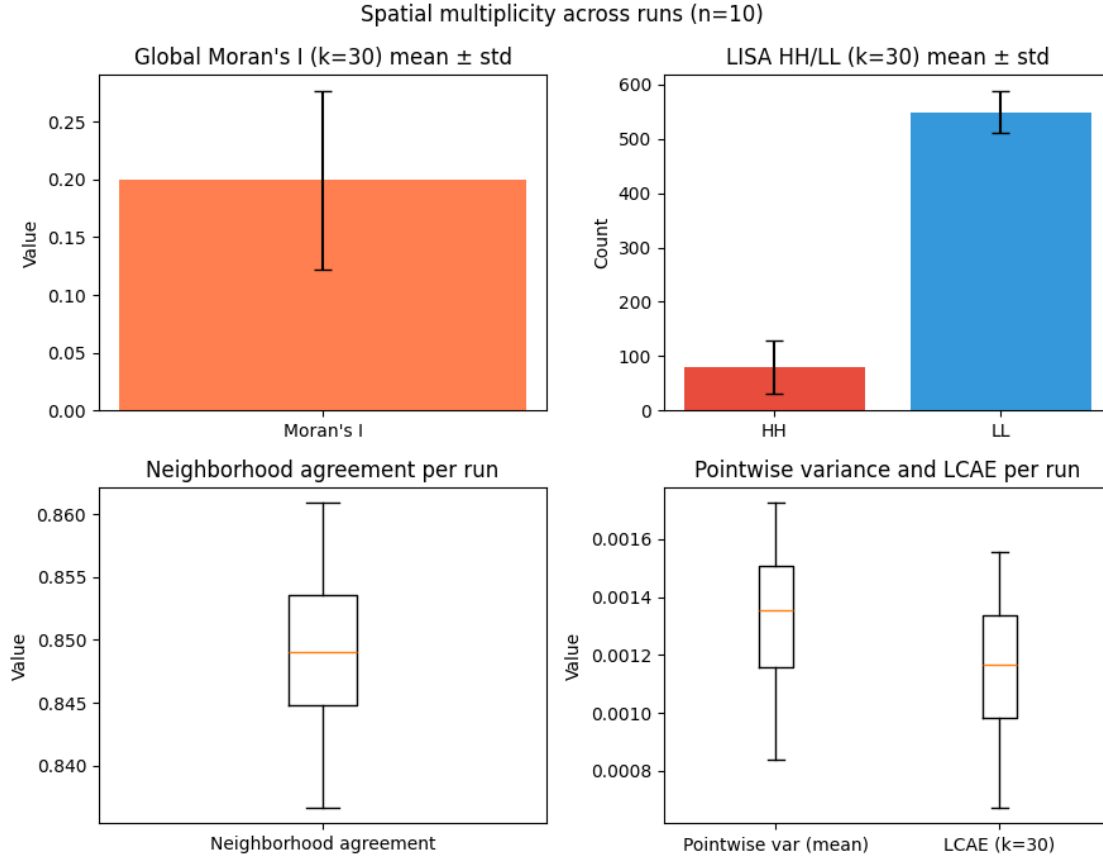
Pointwise variance (already above), Moran's I ($k=30$), LISA ($k=30$), neighborhood agreement, LCAE ($k=30$): local average of pointwise variance over k NN neighbors.

```
[47]: spatial_cols = ["moran_i", "n_hh", "n_ll", "neighborhood_agreement", "lcae",
    ↪ "pointwise_var_mean"]
spatial_agg = df_runs[spatial_cols].agg(["mean", "std"]).T.round(4)
spatial_agg.columns = ["mean", "std"]
spatial_agg["mean ± std"] = spatial_agg.apply(lambda r: f"{r['mean']:.4f} ±
    ↪ {r['std']:.4f}", axis=1)
print("Spatial multiplicity aggregated across runs:")
display(spatial_agg[["mean", "std", "mean ± std"]])

fig, axes = plt.subplots(2, 2, figsize=(9, 7))
ax = axes[0, 0]
ax.bar(["Moran's I"], [df_runs["moran_i"].mean()], yerr=[df_runs["moran_i"].
    ↪ std()], color="coral", capsizes=5)
ax.set_ylabel("Value")
ax.set_title("Global Moran's I (k=30) mean ± std")
ax = axes[0, 1]
ax.bar(["HH", "LL"], [df_runs["n_hh"].mean(), df_runs["n_ll"].mean()],
    ↪ yerr=[df_runs["n_hh"].std(), df_runs["n_ll"].std()], color=["#e74c3c",
    ↪ "#3498db"], capsizes=5)
ax.set_ylabel("Count")
ax.set_title("LISA HH/LL (k=30) mean ± std")
ax = axes[1, 0]
ax.boxplot(df_runs["neighborhood_agreement"], tick_labels=["Neighborhood
    ↪ agreement"])
ax.set_ylabel("Value")
ax.set_title("Neighborhood agreement per run")
ax = axes[1, 1]
ax.boxplot([df_runs["pointwise_var_mean"], df_runs["lcae"]],
    ↪ tick_labels=["Pointwise var (mean)", "LCAE (k=30)"])
ax.set_ylabel("Value")
ax.set_title("Pointwise variance and LCAE per run")
plt.suptitle("Spatial multiplicity across runs (n=%d)" % len(df_runs))
plt.tight_layout()
plt.show()
```

Spatial multiplicity aggregated across runs:

	mean	std	mean ± std
moran_i	0.1993	0.0772	0.1993 ± 0.0772
n_hh	80.7000	48.7375	80.7000 ± 48.7375
n_ll	549.0000	39.0527	549.0000 ± 39.0527
neighborhood_agreement	0.8493	0.0071	0.8493 ± 0.0071
lcae	0.0011	0.0003	0.0011 ± 0.0003
pointwise_var_mean	0.0013	0.0003	0.0013 ± 0.0003



1.6 Summary table (all metrics)

Single table combining performance, predictive multiplicity, and spatial metrics for reporting.

```
[48]: def _mean_std(s):
    m, s_ = s.mean(), s.std()
    return f"{m:.4f}  $\pm$  {s_:.4f}" if len(s) > 1 else f"{m:.4f}"

report_rows = [
    ("Performance", "Accuracy (mean over Rashomon)",
    ↪ _mean_std(df_runs["acc_mean"])),
    ("Performance", "Brier (mean over Rashomon)",
    ↪ _mean_std(df_runs["brier_mean"])),
    ("Performance", "Accuracy (ensemble)", _mean_std(df_runs["acc_ensemble"])),
    ("Performance", "Brier (ensemble)", _mean_std(df_runs["brier_ensemble"])),
    ("Predictive multiplicity", "Mean variance",
    ↪ _mean_std(df_runs["mean_variance"])),
    ("Predictive multiplicity", "Ambiguity", _mean_std(df_runs["ambiguity"])),
    ("Predictive multiplicity", f"Disagreement rate (={EPSILON})",
    ↪ _mean_std(df_runs["disagreement_rate"])),
```

```

    ("Predictive multiplicity", "Discrepancy",
↪_mean_std(df_runs["discrepancy"])),
    ("Spatial multiplicity", "Pointwise variance (mean)",
↪_mean_std(df_runs["pointwise_var_mean"])),
    ("Spatial multiplicity", "Moran's I (k=30)", _mean_std(df_runs["moran_i"])),
    ("Spatial multiplicity", "LISA HH count (k=30)",
↪_mean_std(df_runs["n_hh"])),
    ("Spatial multiplicity", "LISA LL count (k=30)",
↪_mean_std(df_runs["n_ll"])),
    ("Spatial multiplicity", "Neighborhood agreement",
↪_mean_std(df_runs["neighborhood_agreement"])),
    ("Spatial multiplicity", "LCAE (k=30)", _mean_std(df_runs["lcae"])),
]
report = pd.DataFrame(report_rows, columns=["Category", "Metric", "Value (mean
↪± std over runs)"])
print(f"Dataset: {DATASET}, n_runs={len(df_runs)}, K={K}, k_nn={K_NN}")
display(report)

```

Dataset: compas, n_runs=10, K=25, k_nn=30

	Category	Metric \
0	Performance	Accuracy (mean over Rashomon)
1	Performance	Brier (mean over Rashomon)
2	Performance	Accuracy (ensemble)
3	Performance	Brier (ensemble)
4	Predictive multiplicity	Mean variance
5	Predictive multiplicity	Ambiguity
6	Predictive multiplicity	Disagreement rate (=0.05)
7	Predictive multiplicity	Discrepancy
8	Spatial multiplicity	Pointwise variance (mean)
9	Spatial multiplicity	Moran's I (k=30)
10	Spatial multiplicity	LISA HH count (k=30)
11	Spatial multiplicity	LISA LL count (k=30)
12	Spatial multiplicity	Neighborhood agreement
13	Spatial multiplicity	LCAE (k=30)

	Value (mean ± std over runs)
0	0.6796 ± 0.0080
1	0.2092 ± 0.0038
2	0.6821 ± 0.0084
3	0.2079 ± 0.0038
4	0.0013 ± 0.0003
5	0.1249 ± 0.0129
6	0.2552 ± 0.0601
7	0.0651 ± 0.0074
8	0.0013 ± 0.0003
9	0.1993 ± 0.0772
10	80.7000 ± 48.7375

11	549.0000 ± 39.0527
12	0.8493 ± 0.0071
13	0.0011 ± 0.0003